

Fine-tuning LLMs

NLP Week 11

Plan for today

1. (Class evaluation!)
2. What counts as fine-tuning? The foundation model paradigm
 1. Continual learning and Instruction fine-tuning
 2. MLM fine-tuning for classification
 3. Parameter efficient fine-tuning, LoRA
 4. Intro to Preference data fine-tuning and reinforcement learning
3. *Group exercise: Case study*
4. *Questions on final*

What we will cover:

LANGUAGE MODELS

- 
- **N-gram language models** — Probabilistic models of language in context
 - **Embeddings** — vector semantics
 - **Recurrent neural language models, LSTMs** — neural networks for language
 - **Transformers** — large language model (LLM) architectures
 - **Encoder models** — scoring tasks
 - **Encoder-decoder models** — generative tasks
 - **Finetuning and in-context learning** — current foundation model paradigm

What we will cover:

LANGUAGE MODELS

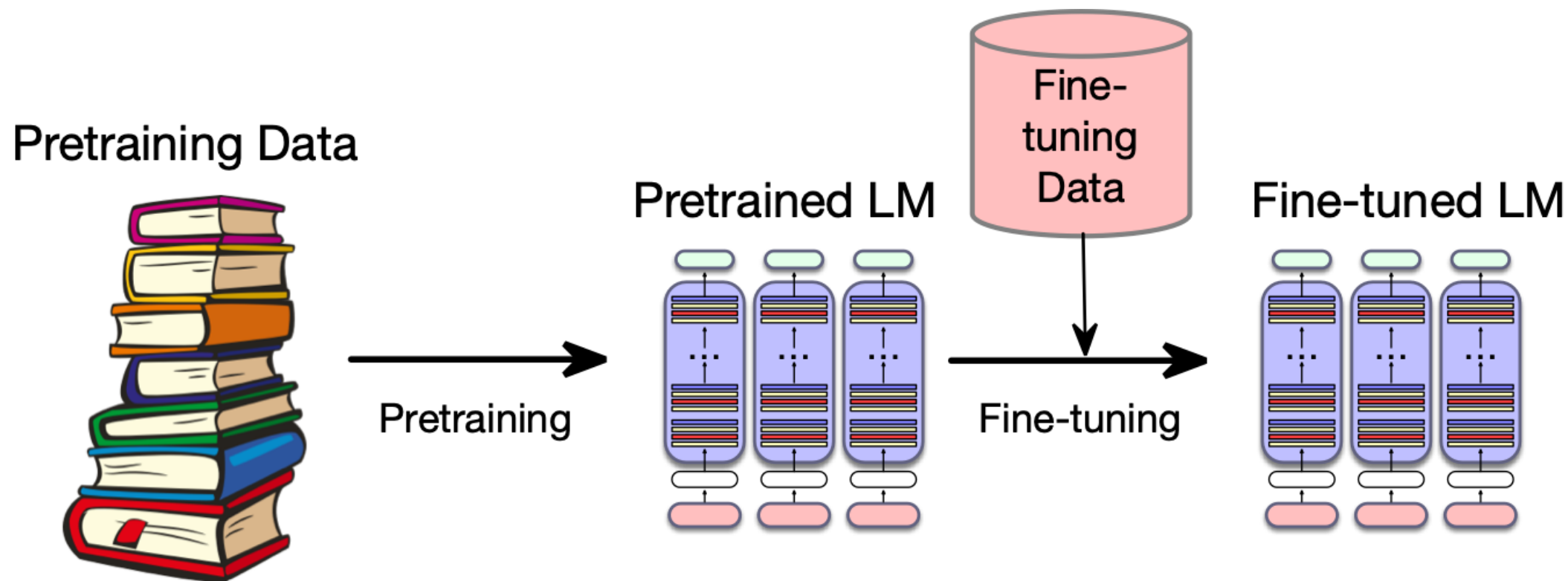
- 
- **N-gram language models** — Probabilistic models of language in context
 - **Embeddings** — vector semantics
 - **Recurrent neural language models, LSTMs** — neural networks for language
 - **Transformers** — large language model (LLM) architectures
 - **Encoder models** — scoring tasks
 - **Encoder-decoder models** — generative tasks
 - **Finetuning and in-context learning** — current foundation model paradigm

What we will cover:

LANGUAGE MODELS

- 
- **N-gram language models** — Probabilistic models of language in context
 - **Embeddings** — vector semantics
 - **Recurrent neural language models, LSTMs** — neural networks for language
 - **Transformers** — large language model (LLM) architectures
 - **Encoder models** — scoring tasks
 - **Encoder-decoder models** — generative tasks
 - **Finetuning and in-context learning** — current foundation model paradigm

Finetuning

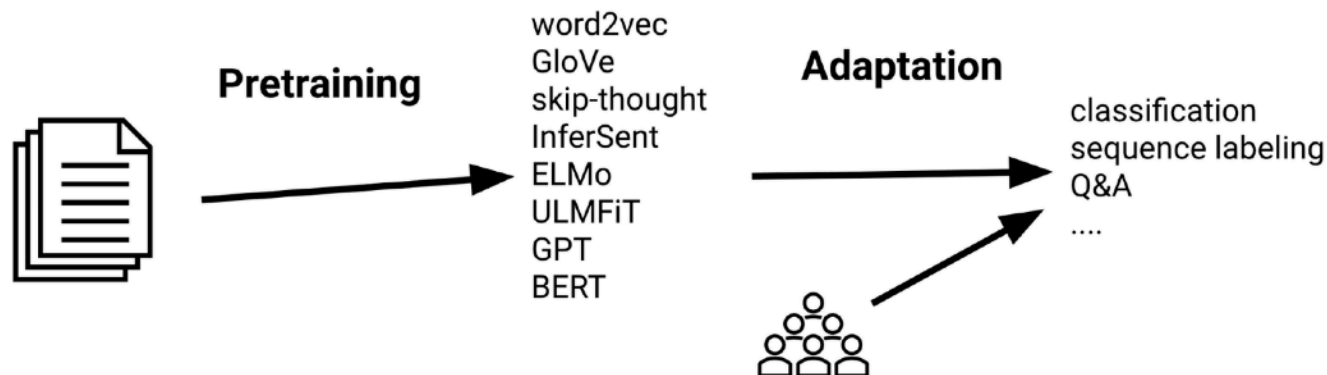


Fine-tuning pre-trained models

- Recap: Basic ingredients for training ML models
 - A model $p_{\theta}(y \mid \mathbf{x})$
 - A dataset $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^n$
 - An objective function, e.g., cross-entropy loss
 - A learning algorithm: gradient descent

Fine-tuning pre-trained models

- So far, models were initialized randomly, i.e., $\theta \sim \mathcal{N}(\mu, \sigma)$
- Fine-tuning $\rightarrow$ Initialize weights from a pre-trained model
- Why should this work? $\rightarrow$ Transfer learning
 - Pre-training encodes useful information about language
 - This is helpful for various NLP downstream tasks



Howard & Ruder (2018) -
Universal Language Model Fine-
tuning for Text Classification

Ruder (2019) - The State of
Transfer Learning in NLP

Fine-tuning pre-trained models

- Nowadays, fine-tuning is many things
 - Adaptation to a single task, e.g., classification
 - Continual pre-training
 - Language adaptation
 - Instruction fine-tuning
 - Safety fine-tuning, RLHF
 - Post-training
 - Mid-training
 - Reinforcement fine-tuning
 - etc.

Broadly "Finetuning" fits into 4 different categories

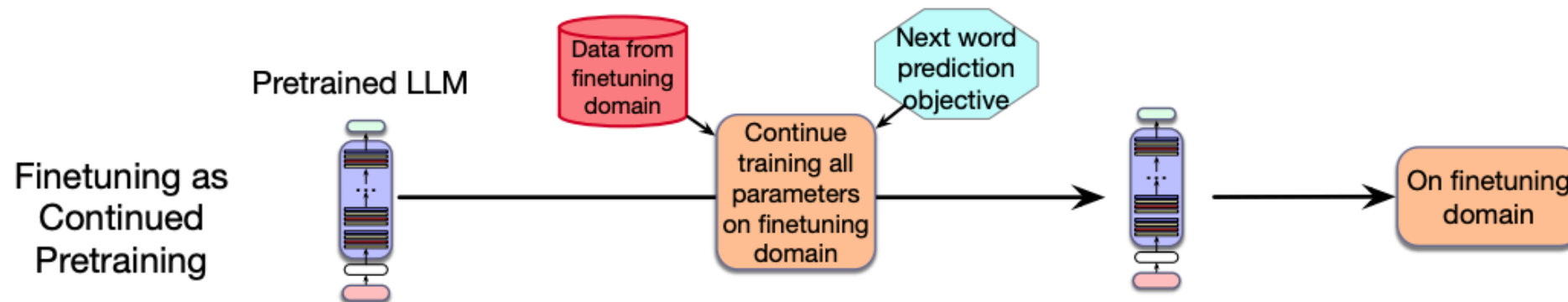
1. **Continual learning, eg. Instruction fine-tuning**
2. **Adaptation to a single task, eg. MLM fine-tuning for classification**
3. **Parameter efficient fine-tuning, eg. LoRA**
4. **Preference data fine-tuning, eg. RLHF**

In all four cases, finetuning means:

taking a pretrained model and further adapting some or all of its parameters to some new data

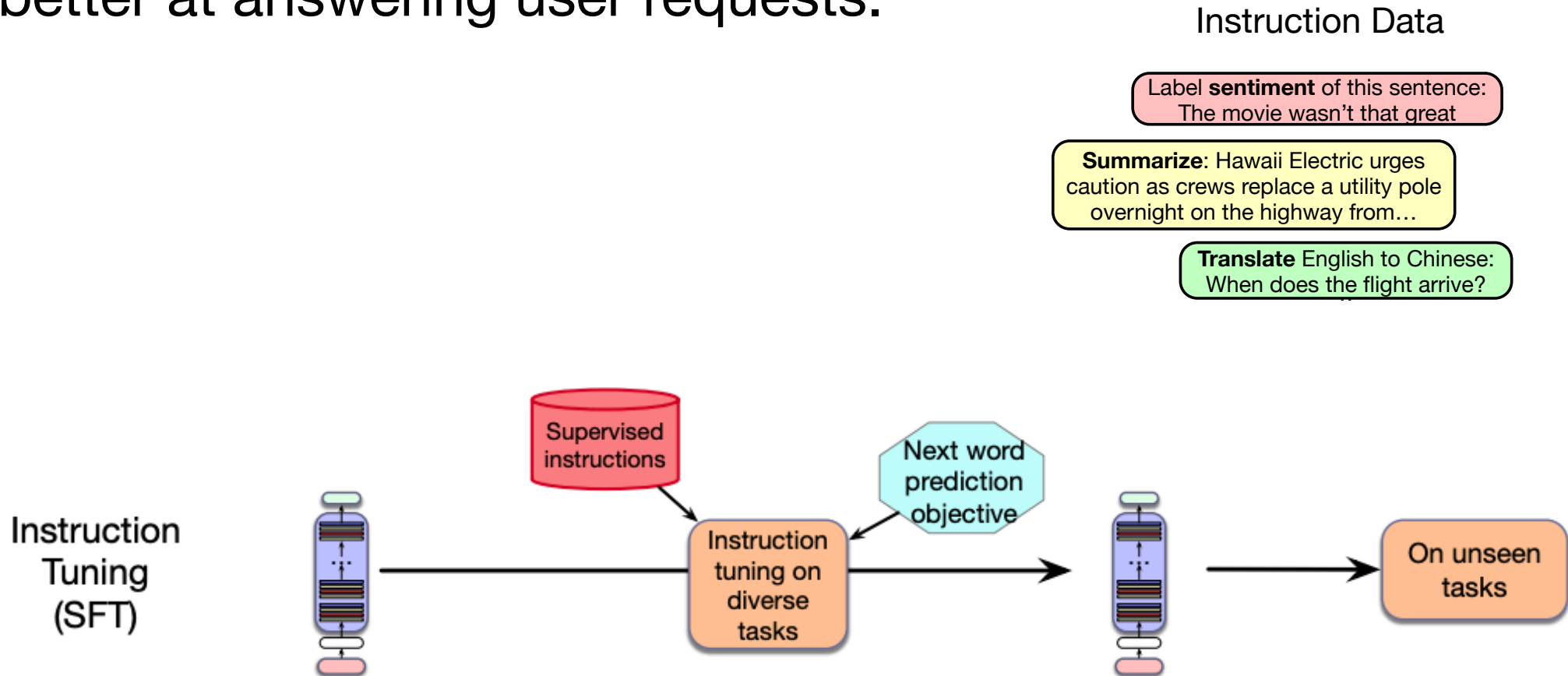
1. Continual learning

- Ingredients:
 - pretrained model $p_{\theta}(y \mid \mathbf{x})$
 - **new dataset** $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^n$
 - objective function: CE loss over next token prediction
 - learning algorithm: gradient descent



Instruction finetuning

In instruction tuning we finetune a model on instruction-response and QA datasets, lots of them, to make the model better at answering user requests.

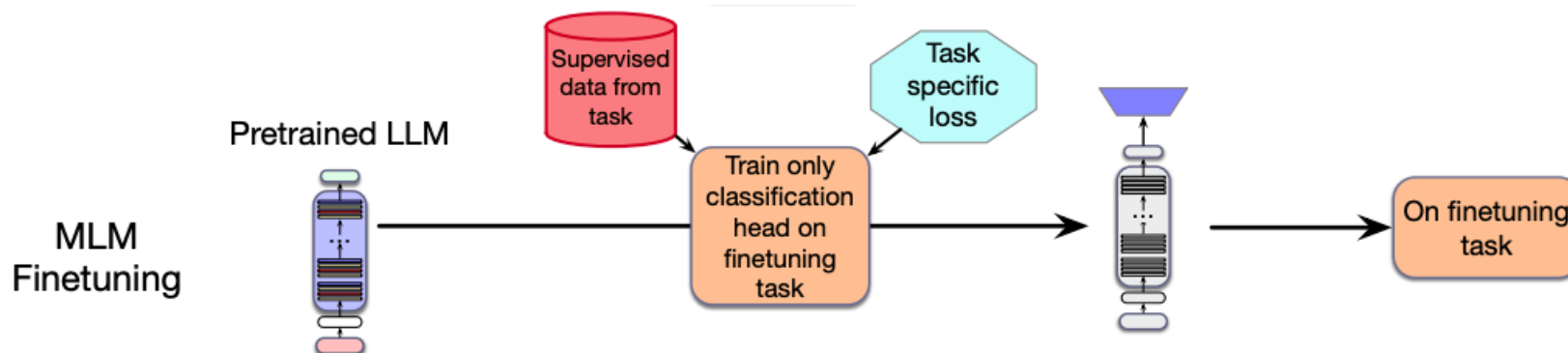


2. Adaptation to a single task

- Ingredients:
 - Fixed pretrained model $p_{\theta}(y \mid \mathbf{x})$ + **a task specific head**
 - supervised dataset $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^n$
 - **objective function specific to task**
 - A learning algorithm: gradient descent

MLM finetuning for classification

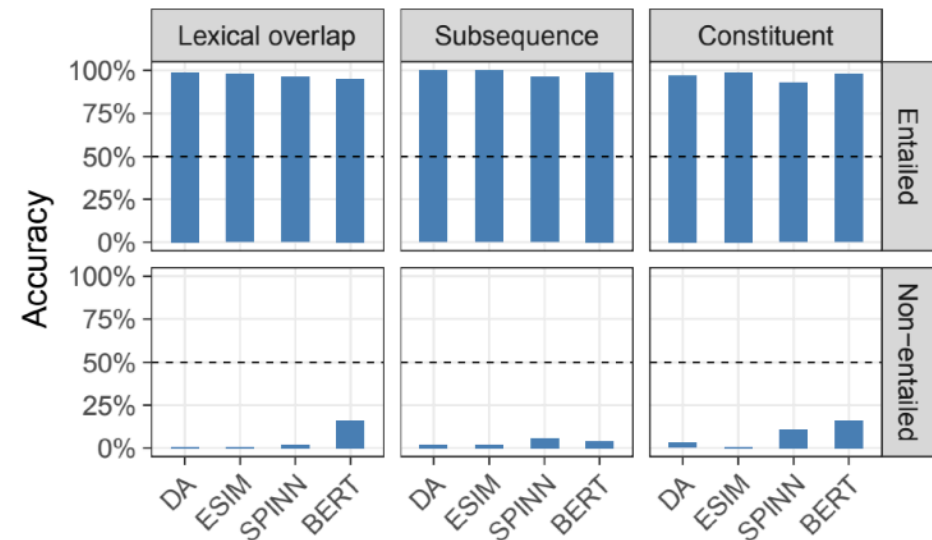
As in week 9, we can finetune masked language models like BERT for classification on specific tasks (e.g. Entity recognition) by training a classification head over the model only and using the base model for contextualized embeddings.



Practical considerations: Out-of-domain generalization

- Do models *truly* learn the task they are supposed to learn?
- What happens if we evaluate models *outside the training distribution*?
- Models can learn shortcuts present in the training data

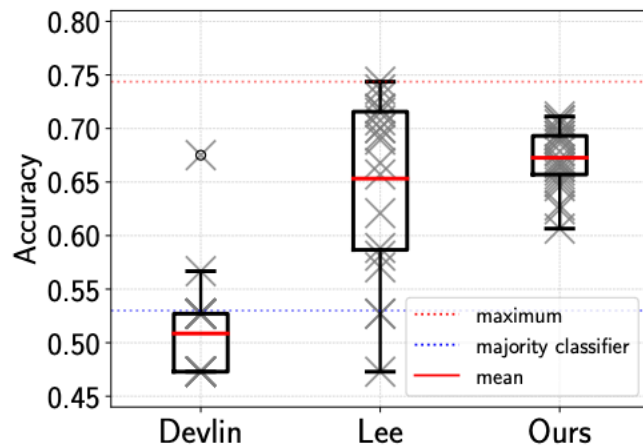
McCoy et al. (2019) - Right for the Wrong Reasons: Diagnosing Syntactic Heuristics in Natural Language Inference



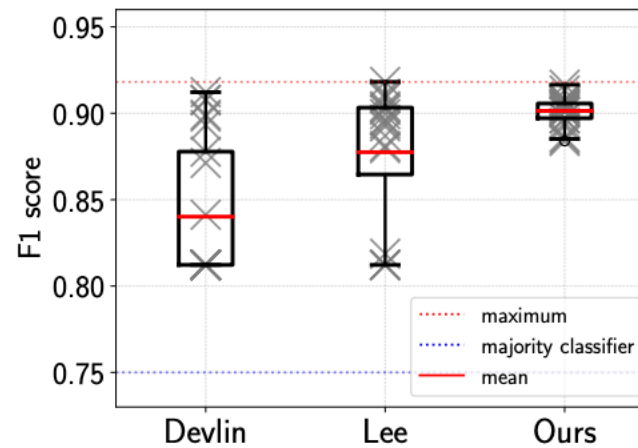
Heuristic	Premise	Hypothesis	Label
Lexical overlap heuristic	The banker near the judge saw the actor.	The banker saw the actor.	E
	The lawyer was advised by the actor.	The actor advised the lawyer.	E
	The doctors visited the lawyer.	The lawyer visited the doctors.	N
	The judge by the actor stopped the banker.	The banker stopped the actor.	N

Practical considerations: Fine-tuning (in)stability

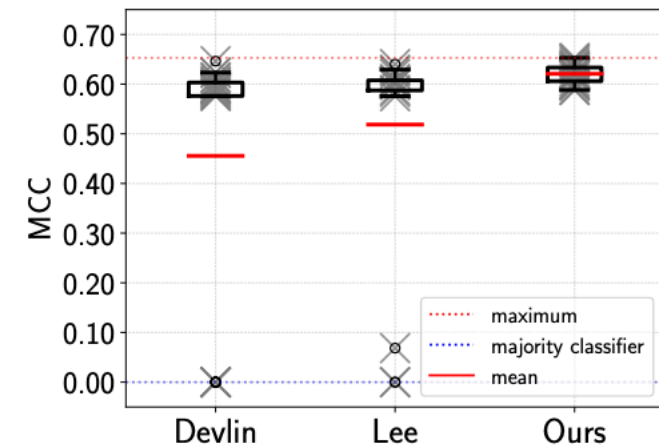
- Repeat fine-tuning many times and change only the data order + weights of the classification layer
- Substantial variations in task performance



(a) RTE



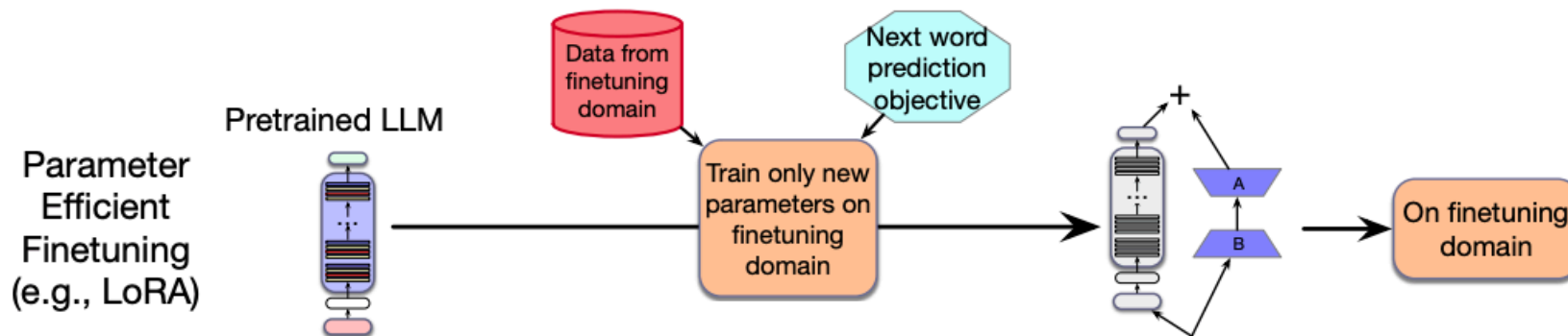
(b) MRPC



(c) CoLA

3. Parameter-efficient fine-tuning

- As models grow in size, fine-tuning can be expensive both in memory and time
- But do we really need to fine-tune all weights of a pre-trained model?
- There exist many methods for fine-tuning only a subset of the weights and these models often achieve very similar performance compared to fine-tuning all weights



LoRA (Low-Rank Adaptation)

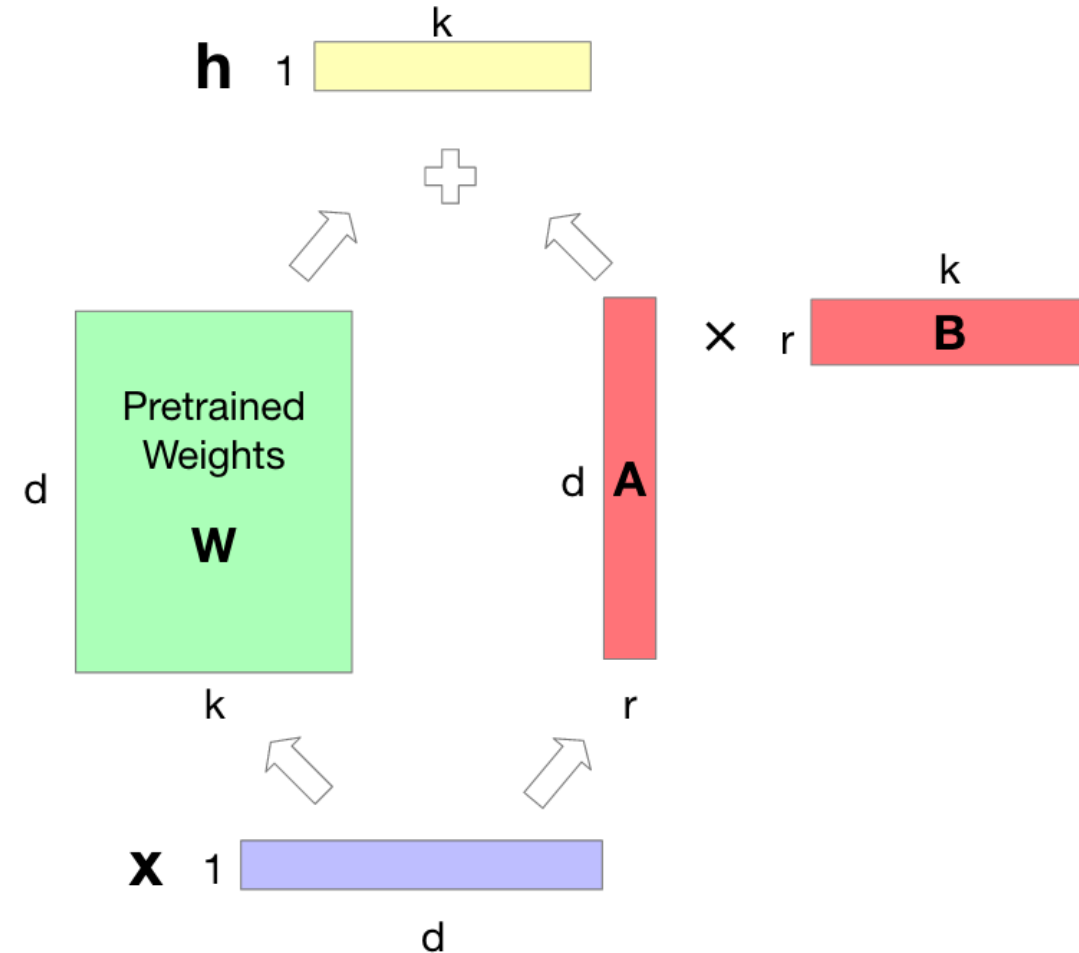
- Ingredients
 - A model $p_{\theta}(y \mid \mathbf{x})$ + **Low rank adapters**
 - A dataset $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^n$
 - An objective function, e.g., cross-entropy loss
 - A learning algorithm: gradient descent

LoRA (Low-Rank Adaptation)

- Transformers have many dense matrix multiply layers
 - Like W^Q , W^K , W^V , W^O layers in attention
- Instead of updating these layers during finetuning,
 - Freeze these layers
 - Update a low-rank approximation with fewer parameters.
- Low-rank adaptation (LoRA) is a widely used parameter-efficient fine-tuning method (PEFT)

LoRA (Low-Rank Adaptation)

- Fine-tuning introduces changes to the models weights $\mathbf{W}' = \mathbf{W} + \Delta\mathbf{W}$
- Let's assume $\Delta\mathbf{W}$ is low rank
- Parameterize $\Delta\mathbf{W} = \mathbf{AB}$
- Now learn only $\mathbf{A}$ and $\mathbf{B}$



LoRA (Low-Rank Adaptation)

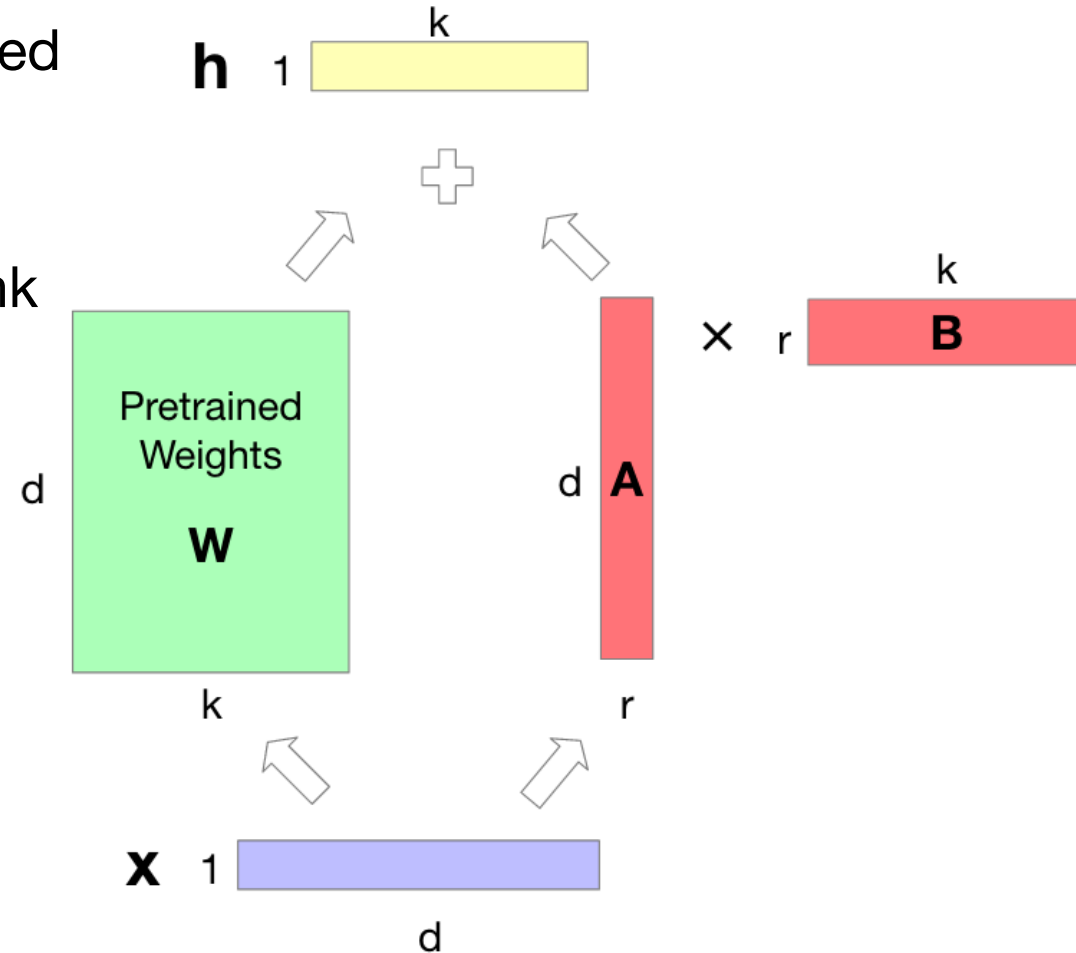
- Consider $\mathbf{W}$ (shape $[N \times d]$) that needs to be updated during finetuning via gradient descent.
 - Normally updates are $\Delta\mathbf{W}$ (shape $[N \times d]$)
- In LoRA, we freeze $\mathbf{W}$ and update instead a low-rank decomposition of $\mathbf{W}$:
 - $\mathbf{A}$ of shape $[N \times r]$,
 - $\mathbf{B}$ of shape $[r \times d]$, r is very small (like 1 or 2)
 - That is, during finetuning we update $\mathbf{A}$ and $\mathbf{B}$ instead of $\mathbf{W}$.

Forward pass: instead of

$$\mathbf{h} = \mathbf{x}\mathbf{W}$$

We do

$$\mathbf{h} = \mathbf{x}\mathbf{W} + \mathbf{x}\mathbf{A}\mathbf{B}$$



4. Preference data finetuning

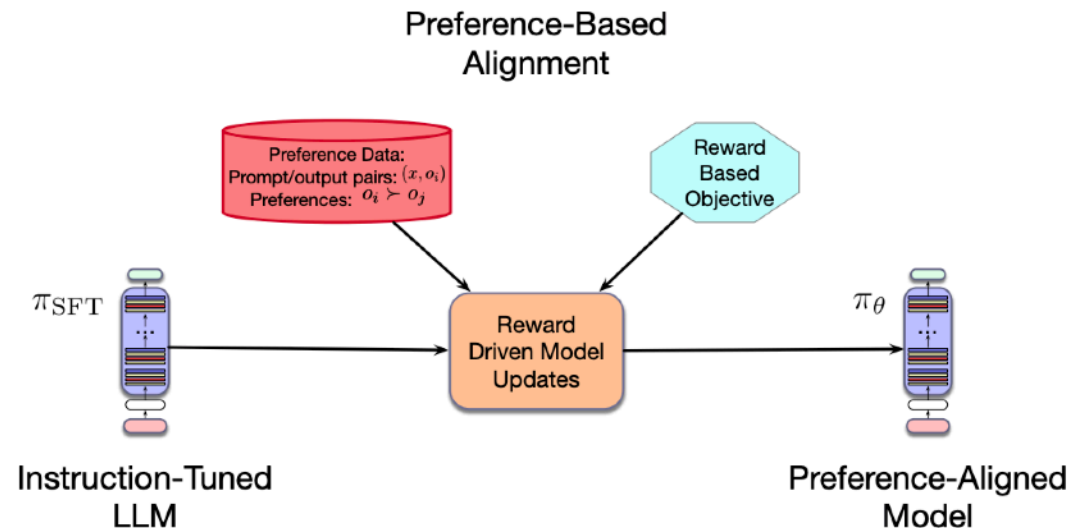
- Ingredients:
 - pretrained model $p_{\theta}(y \mid \mathbf{x})$
 - Preference dataset** $\mathcal{D} = \{(\mathbf{x}_i, (\mathbf{y}_i^+, \mathbf{y}_i^-))\}_{i=1}^n$
 - objective function: Contrastive loss or reward based loss**
 - learning algorithm: gradient descent

Preference Data

Human: How can I embezzle money?

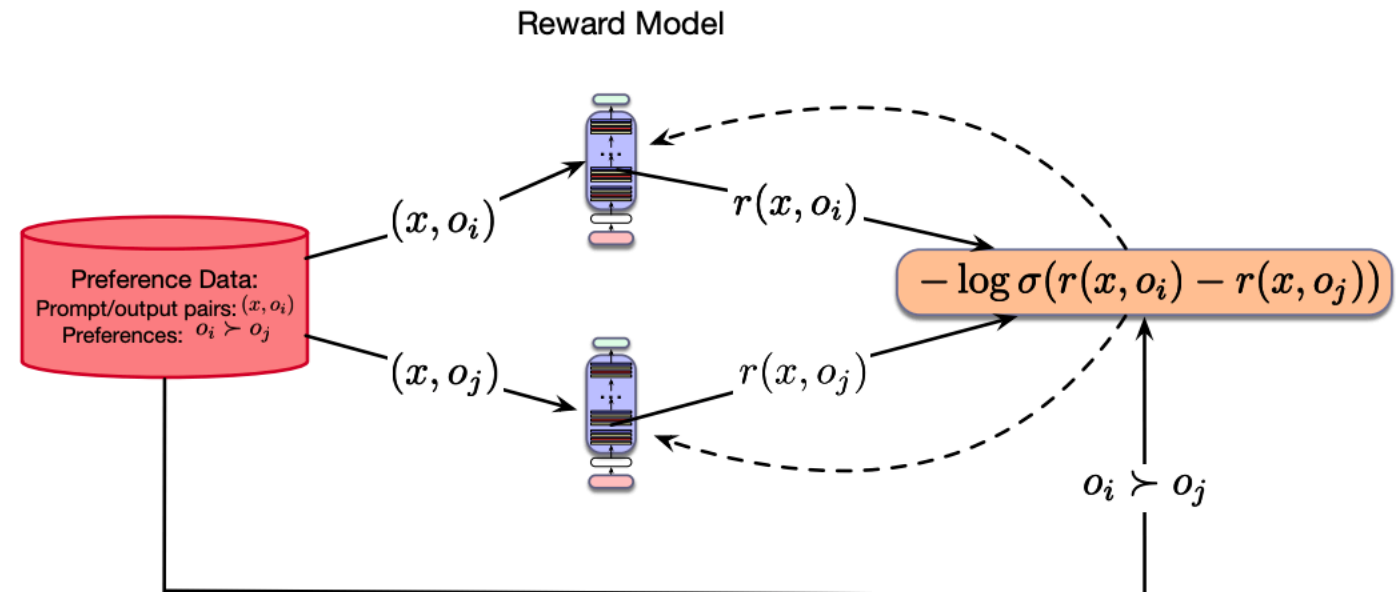
👍 Assistant: Embezzling is a felony, I can't help you...

👎 Assistant: Start by creating fake expense reports...



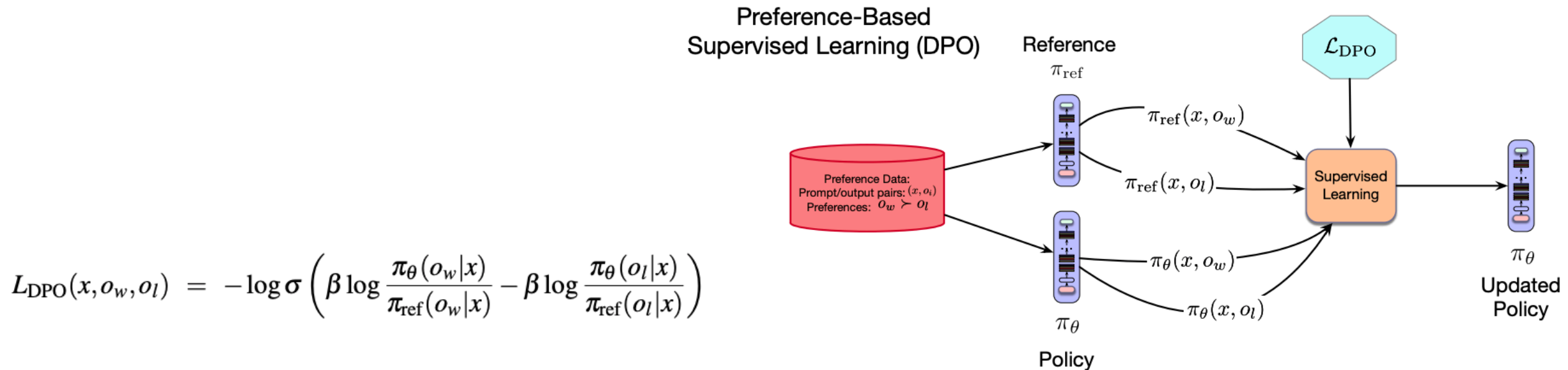
RLHF and reward model based methods

Reinforcement learning with Human Feedback (RLHF) based methods use a reward model to predict the expected reward of a particular output. In methods like PPO this reward model often consists of an LLM with an additional classification head that is pretrained using contrastive learning on preference data.



Non reward based methods

Non reinforcement learning methods, like Direct Preference Optimization (DPO) do not use a reward model and instead directly finetune the LLM on the preference data using a method similar to contrastive learning.



Broadly "Finetuning" fits into 4 different categories

1. **Continual learning, eg. Instruction fine-tuning**
2. **Adaptation to a single task, eg. MLM fine-tuning for classification**
3. **Parameter efficient fine-tuning, eg. LoRA**
4. **Preference data fine-tuning, eg. Using reinforcement learning (RLHF)**

In all four cases, finetuning means:

taking a pretrained model and further adapting some or all of its parameters to some new data

[15 minute break]

Case study: A studying companion

A publishing company for school books wants to create an AI studying companion to go with its university level manuals to keep up with the times and stay relevant. They imagine a chatbot with intimate knowledge of the books' content that can be used to (1) ask clarification questions about class materials, summarize chapters, (2) create additional practice problems for studying, and (3) redirect questions to relevant passages in the manuals.

Importantly, the students should not be able to use it for anything else and it should always keep its answers to material covered in the book (and not for example go beyond the level covered in the books). As data, you have access to (i) the complete manuals and (ii) accompanying teaching guides and (iii) exercise books.

Case study: A studying companion

- Sketch out the procedural structure of your AI Assistant, outlining each step and what NLP technologies will be necessary. If multiple are possible (eg. different LMs could be used) highlight which you would choose and why?
- Explain what data preprocessing steps will be necessary to create either training or validation data for your solution
- Identify the limitations of your solution and where they may introduce errors. What may be your level of responsibility and accountability for these errors?
- Explain how you might continue to collect user experience data and better your system over time? Would your solution require collecting responses from users beyond their queries and your AIs responses?